Blink: Neues Bild

Lernunterlage: eine bewusste Ausnahme von „nur lesend“ – Admin-Recht, Rate Limit pro Kamera, Befehl abfragen

1 Worum es geht

Blink erneuert das Vorschaubild nur bei einer Bewegung. Ohne Bewegung zeigt die Kamerakarte ein Bild, das Tage alt sein kann. Neu ist deshalb ein Knopf **„Neues Bild“**: Die Kamera wacht auf und nimmt ein Standbild auf, wie „Bild aktualisieren“ in der Blink-App.

Bisher galt streng: Tuya und Blink sind **nur lesend**. Der Knopf ist die **einzige bewusste Ausnahme**. Er schaltet nichts (nicht scharf/unscharf, kein Clip, keine Einstellung), kostet aber **Akku**, weil die Kamera aufwacht und über WLAN sendet. Deshalb ist er eng begrenzt und dokumentiert (BLINK.md, CLAUDE.md).

> Eine Ausnahme von einer Regel ist in Ordnung, wenn sie bewusst entschieden, eng begrenzt und aufgeschrieben ist. Sonst wird aus der Ausnahme schleichend die neue Regel.

2 Drei Schranken

Schranke	Umsetzung	Warum
Wer?	nur Admin (Policy AdminOnly, JWT)	User 403, KI-Agent 401: Akku ist teuer
Wann?	nur auf Knopfdruck	kein Hintergrunddienst löst Bilder aus
Wie oft?	1× pro Minute und Kamera (429)	auch ein Doppelklick leert den Akku nicht

```
blink.MapPost("/cameras/{id:long}/snapshot", RequestSnapshot)
    .RequireAuthorization(AuthPolicies.AdminOnly)            // nur Admin
    .RequireRateLimiting(RateLimitPolicies.BlinkSnapshot);    // 1× pro Minute und Kamera
```

POST statt GET: GET heißt in HTTP „nur lesen, beliebig oft wiederholbar“. Ein neues Bild verändert etwas (Akku, neues Bild bei Blink), deshalb POST. Browser und Proxys wiederholen POST nie von selbst.

3 Rate Limit pro Kamera statt pro Gerät

Die anderen Regeln zählen pro Client-IP (Login, Bilder) oder gemeinsam (Tuya). Hier geht es um den Akku **einer Kamera**. Also wird nach der Kamera-Id aus der Adresse gezählt:

```
options.AddPolicy(RateLimitPolicies.BlinkSnapshot, context =>
{
    var limit = GetOptions(context).BlinkSnapshot;                // Standard 1 pro 60 s
    var camera = context.Request.RouteValues["id"]?.ToString() ?? "unbekannt";
    return RateLimitPartition.GetFixedWindowLimiter(camera, _ => CreateFixedWindow(limit));
});
```

- **Partition:** Jede Kamera bekommt ihren eigenen Zähler. Wohnzimmer gesperrt heißt nicht Flur gesperrt.

- **Egal von wo:** Handy und Laptop teilen sich den Zähler derselben Kamera. Pro IP wäre das nicht so.
- **429 mit Retry-After:** Die Oberfläche zeigt „Neues Bild für diese Kamera höchstens einmal pro Minute. Bitte in … erneut versuchen.“

4 Befehl abfragen statt blind warten

Blink antwortet auf den POST sofort mit einer **Befehls-Id**. Das Bild ist da noch nicht fertig: Die Kamera muss erst aufwachen. Das Backend fragt deshalb den Befehlsstatus ab (Polling):

1. POST an Blink: `/network/{n}/camera/{id}/thumbnail` (Mini: `…/owls/{id}/thumbnail`, Türklingel: `…/doorbells/{id}/thumbnail`, wie im Projekt blinkpy).
2. Antwort lesen: `{ "id": 555 }`.
3. Jede Sekunde `GET /network/{n}/command/555`, bis `complete: true`, höchstens 40-mal.
4. Homescreen und Bild-Cache verwerfen, neu holen: neuer Bildpfad, neue Aufnahmezeit.

```
for (var i = 0; i < MaxCommandPolls; i++)                  // höchstens 40-mal
{
    // ... GET /network/{n}/command/{id}, complete == true -> fertig
    await Task.Delay(CommandPollInterval, timeProvider, ct);  // im Test: TimeSpan.Zero
}
```

- **Obergrenze:** Ohne die 40 würde eine hängende Kamera die Anfrage ewig offen halten.
- **completed: false** ist kein Absturz: Blink hat den Befehl angenommen, nur nicht rechtzeitig gemeldet. Die Karte sagt das ehrlich.
- **Gescheitert:** Meldet Blink einen anderen `status_code` als 908 („läuft noch“), wird nicht weiter gewartet.
- **Token:** Lehnt Blink den POST mit 401 ab, wird einmal erneuert, dann gibt es einen Fehler statt einer Schleife. Dieselbe Regel wie bei den anderen Blink-Aufrufen.

5 Im Frontend

- Den Knopf sehen nur Admins (`isAdmin` aus `useAuth()`). Das ist nur Bequemlichkeit: **Die echte Prüfung macht das Backend.**
- Während der Aufnahme steht „Kamera nimmt auf …“ und der Knopf ist gesperrt (kein Doppelklick).
- Danach zählt ein Zähler (`snapshotCount`) hoch. Er steht in den Abhängigkeiten des Effekts, der das Bild lädt. So lädt React das neue Bild, ohne eigenen Sonderweg.
- Die **Aufnahmezeit** steht groß unter dem Bild, **gelb**, wenn sie älter als einen Tag ist. Ein Vorschaubild ist kein Livebild.

6 Tests

Test	prüft
Neues_Bild_loest_POST_aus_wartet_auf_den_Befehl_…	richtige Adresse, POST, unser Token, neue Aufnahmezeit
Neues_Bild_fuer_eine_Mini_nimmt_die_owls_Adresse	Mini-Kamera bekommt die andere Adresse
Neues_Bild_gibt_nach_40_Abfragen_auf	keine Endlosschleife
Neues_Bild_fuer_unbekannte_Kamera_loest_nichts_aus	kein Aufruf bei Blink

Test	prüft
Neues_Bild_als_User_403_mit_Agenten_Schluessel_401	nur Admin
Neues_Bild_als_Admin_hoechstens_einmal_pro_Minute_…	zweiter Klick 429 mit Retry-After, andere Kamera 200

Die Blink-Cloud wird im Test nie aufgerufen: Ein Fake-`HttpMessageHandler` spielt POST und Befehlsstatus. Live ausprobiert wird erst nach dem Merge im Dienst, weil jeder Versuch Akku kostet.

7 Kurzantworten für Rückfragen

Frage	Antwort
Ist das nicht Schalten?	Nein: nichts wird ein- oder ausgeschaltet. Es kostet nur Akku, deshalb begrenzt.
Warum POST?	Es verändert etwas. GET muss gefahrlos wiederholbar sein.
Warum pro Kamera zählen?	Geschützt wird der Akku einer Kamera, egal von welchem Gerät geklickt wird.
Warum nur Admin?	Teure Aktion; der KI-Agent und normale User dürfen nur lesen.
Was ist Polling?	Wiederholt nachfragen, bis etwas fertig ist, mit Pause und Obergrenze.
Warum Knopf verstecken, wenn das Backend prüft?	Nur Bequemlichkeit. Sicherheit kommt immer vom Server.

8 Merksätze

- Ausnahmen bewusst entscheiden, eng begrenzen, aufschreiben.
- Verändernde Aufrufe sind POST, nie GET.
- Rate Limit nach dem zählen, was geschützt wird (hier: die Kamera).
- Polling immer mit Pause und Obergrenze.
- Knöpfe ausblenden ist Komfort, die Rechte prüft der Server.